Bug Report Classification Tool Building Report – Tanisha Bhogal – 2426900

1. Introduction

As software systems grow in complexity, the number of bug reports submitted to issue tracking systems increases exponentially, making manual triage a significant challenge. Detecting performance bugs early is vital for maintaining software quality and user experience. However, it remains a significant problem because bug reports raised around performance issues can be vague and difficult to triage. Many bug reports will contain a mix of both natural language and code snippets, which can be tricky to semantically handle.

The baseline approach for this task utilises Term Frequency-Inverse Document Frequency (TF-IDF) combined with a Naive Bayes classifier. However, it relies exclusively on the natural language found in the title and body of these reports, excluding any provided code snippets, stack traces or comments. By treating only a fraction of the report as flat text, it fails to capture the structural nuances and context required to identify performance bugs.

To address these limitations, this report proposes a multifaceted Logistic Regression pipeline that explicitly separates the semantic evaluation of natural language (via Latent Semantic Analysis) from the lexical evaluation of rigid code snippets, while incorporating structured metadata. Following a review of the relevant literature, the proposed architecture is detailed and evaluated against the baseline. Experimental results across five datasets demonstrate that this structurally aware approach significantly outperforms the baseline, consistently beating the Accuracy, F1-Score and Area Under the Curve (AUC) value across all datasets. This demonstrates that it is a robust, explainable, intelligent software engineering tool that can handle automated bug triage.

2. Related work

Several studies and experiments have been conducted to determine how to most effectively classify bugs in an automated fashion.

We can see that early foundational work in automated bug classification relied heavily on purely lexical approaches. Antoniol et al. [1] investigated the use of decision trees, Naïve Bayes, and logistic regression to distinguish bugs from other issue types. Their work established a baseline for classification, utilising text pre-processing and raw term frequencies to classify bugs. Initially, these raw frequencies were gathered across several extracted features, including the title, description and discussion. The team later concluded that using the sole issue description yielded the best results.

Javed et al. [2] took a similar approach, utilising TF-IDF and chi-square algorithms to extract features and Naïve Bayes to classify the reports (which links to the baseline approach for this task). The future work section notes that other feature selection techniques could improve results.

While these approaches provide a solid lexical baseline, both fail to account for the inherent vagueness of natural language. As demonstrated in the empirical study by Nistor et al. [3], the natural language used in bug reports – particularly for performance issues – can be highly subjective, exposing the limitations of relying solely on exact word matches. Javed et al. [2] actually note this limitation in their future work section, suggesting that a synonym dictionary would further improve the quality of classification.

To address the limitations of purely lexical models, subsequent research integrated semantic analysis. Chawla et al. [4] addressed the 'Out-of-Vocabulary' (OOV) problem by combining TF-IDF with Latent Semantic Indexing (LSI). This hybrid approach successfully captured underlying contexts and synonyms, mitigating the challenges of natural language variation. Yet, while it improved semantic understanding, it still treated bug reports as flat text, as pre-processing techniques and tokenisation was only applied to report titles (ignoring the overall semi-structured nature of the report).

More recently, the field has turned to deep learning to address both semantic and structural challenges. Ramay et al. [5] proposed a deep neural network for severity prediction, utilising vector

embeddings and sentiment analysis. Similarly, Mani et al. [6] utilised unsupervised Recurrent Neural Networks (RNNs) to improve semantic understanding of natural language. This report also acknowledged the messy and noisy nature of bug reports, though pre-processing was primarily used to filter this out. While these approaches successfully capture complex semantic and syntactic features, they introduce a critical flaw for practical software engineering: deep learning models act as opaque 'black boxes.' In the event of a misclassification, they lack the transparency required for engineers to understand the model's reasoning.

The proposed solution in this report is fundamentally inspired by the multifaceted approach of Long et al. [7], which explicitly separates key report components (e.g., title, description, and code snippets) to extract hierarchical semantic knowledge (though it is worth noting that the best results were produced in the case that the title and description were concatenated).

The methodology set out in this report is rooted in this multifaceted and semantic approach, but rather than utilising a complex, opaque deep learning architecture, this project addresses the semi-structured nature of bug reports through explicit feature engineering paired with an explainable classifier. By combining structural awareness with semantic understanding, this approach aims to significantly outperform the baseline while maintaining crucial model transparency.

3. Solution

My proposed solution builds on both the baseline and the various approaches detailed in the related work section. Many of the later articles emphasise that bug reports are inherently multifaceted, and so processing the entire report as one large block of text often destroys necessary context. This is one of the biggest limitations of the baseline approach – it treats the natural language in the title and body the same as the rigid code snippets and stack traces. Fundamentally, this solution adopts the structural approach of Long et al. [7], without the complex, black box, deep learning aspects.

My approach accounts for these various features by running two separate pipelines depending on the part of the bug report being examined. A purely lexical approach is taken for the code and comment sections, by applying TF-IDF to each section in turn, as these are more likely to contain explicit error tokens (unlike the baseline, which chooses to ignore these sections entirely). Attempting to understand this data semantically would not work and destroy the exact error messages needed for classification. Alongside this, I have also extracted the length of the code and comment sections of each report, as we are far more likely to see long, complex logs that detail degradation over time for performance issues. Z-score normalisation (StandardScaler) was used to ensure these large integers could be safely integrated alongside TF-IDF.

For the title and body sections, I have chosen to build on the baseline approach, running a semantic pipeline that combines TF-IDF and TruncatedSVD (Latent Semantic Analysis). Again, the pipeline is run on each section separately. By using Latent Semantic Analysis, I can effectively tackle the OOV problem in a similar fashion to Chawla et al. [4], as it compresses word frequencies into dense 'Latent Topics' and allows the model to recognise mappings between words.

The final key change I have made from the baseline approach is to use Logistic Regression over Gaussian Naïve Bayes. As TF-IDF vectors are usually sparse, Gaussian Naïve Bayes did not perform well on the classification task, struggling to fit the bell curve to this data and leading to poor class separation.

At first, I investigated using Random Forest (a tree-based approach), but the sparse nature of TF-IDF was again the main issue; many of the results produced were worse than the baseline, as the decision trees tend to overfit on sparse data.

Instead, I settled on Logistic Regression. This was chosen because it is highly effective at evaluating mixed feature spaces. It calculates the optimal weights for all features simultaneously, allowing it to combine the information learned from the lexical, semantic and feature engineering approaches into a single probability score. Furthermore, by utilising the class_weight='balanced' hyperparameter,

Logistic Regression dynamically adjusts its loss functions based on the frequencies in the training data and penalises misclassifications of the minority class more heavily, minimising any bias towards the majority class – which in turn improves the overall F1 score, not just the raw accuracy.

4. Setup

The chosen baseline, as outlined above, was TF-IDF with Gaussian Naïve Bayes. Both the baseline and the proposed solution use the datasets provided in Lab 1 (Caffe, Keras, MXNet, PyTorch, and TensorFlow). These datasets contain a mix of functional and performance bugs, with performance-related bugs making up 16.4% of these datasets overall. The baseline utilises a standard TF-IDF vectoriser (configured to remove standard English stop words) on the concatenated Title and Body string, passing the resulting sparse matrix into a standard GaussianNB classifier.

The experimental procedure was designed to ensure that results are rigorous and reproducible. The pipeline and models were implemented in Python, primarily utilising pandas for data manipulation, and the scikit-learn library for text vectorisation, machine learning architectures, and metric evaluation. The data pipeline begins with a data cleaning process, removing html tags, emojis and noisy characters from reports. Structural lengths are also extracted at this stage. For text vectorisation, the lexical TF-IDF pipelines were limited to max_features=1000 to isolate the key technical tokens. Meanwhile, the semantic pipelines were limited to max_features=500 before LSA compression (n_components=20) to ensure the model could identify and correctly group core overarching topics without memorising noisy or project-specific vocabulary. The model was evaluated over 10 iterations using an 80/20 Train-Test split. The data was also stratified to preserve the class imbalance in every test set, to fairly evaluate the model's performance on correctly identifying the performance bugs (the minority class).

The proposed multifaceted Logistic Regression architecture is compared against the provided baseline solution for lab 1. This baseline concatenates the title and body of the bug report into a single flat string, disregarding the code and comment data. It then applies a global TF-IDF vectoriser and utilises a Gaussian Naïve Bayes classifier. Both models were evaluated over 10 iterations using the same 80/20 Train-Test split. The only change made to the provided baseline code was stratifying the Train-Test data to preserve the class imbalance and ensure a fair comparison.

Model performance is evaluated using Accuracy, Precision, Recall, F1-Score and Area Under the Curve (AUC). As the datasets are largely imbalanced overall, Accuracy is not a reliable metric – instead F1 and AUC are utilised as primary indicators of model performance and success.

In terms of statistical analysis, a Two-Tailed Wilcoxon Signed-Rank test was utilised, to ensure that the observed results were reliable. This specific test was selected as the data is paired – both the baseline and proposed solution were evaluated on the exact same data across 10 iterations. As it is a non-parametric test, it does not assume that evaluation metrics follow a Gaussian distribution. Because machine learning metrics like F1-Score and Accuracy are bounded between 0 and 1, they frequently violate the normality assumptions required by parametric alternatives, making the Wilcoxon test an ideal choice. A standard significance threshold of $\alpha = 0.05$ was established to reject the null hypothesis (H_0), which hypothesises that both models perform identically.

5. Experiments

Table 1: Summary of Model Performance across all five datasets

Project	Baseline F1	Proposed F1	Baseline AUC	Proposed AUC
pytorch	0.5522	**0.6235**	0.7465	**0.9041**
tensorflow	0.5434	**0.7063**	0.7164	**0.9330**
keras	0.5737	**0.6823**	0.7163	**0.9117**
incubator-mxnet	0.5456	**0.5642**	0.7423	**0.9012**
caffe	0.4752	**0.5029**	0.6873	**0.8851**

Table 2: Wilcoxon Signed-Rank Test Results (p-values) across all five datasets

Project	F1	AUC	Accuracy
PyTorch	0.105	0.002	0.002
TensorFlow	0.002	0.002	0.002
Keras	0.004	0.002	0.002
MXNet	0.557	0.002	0.002
Caffe	0.322	0.002	0.002

To rigorously assess the proposed solution under diverse scenarios, the models were evaluated across reports for five distinct software systems (Caffe, Keras, MXNet, PyTorch, and TensorFlow). The evaluation considered multiple metric objectives, specifically contrasting raw Accuracy against minority-class identifiers (F1-Score and AUC). Full iteration data is provided in the Appendix, with the statistical summary discussed below.

The proposed multifaceted Logistic Regression model demonstrated dominance over the baseline in both Accuracy and Area Under the Curve (AUC). Across all five datasets, the proposed model consistently achieved a higher Accuracy and AUC in all 10 iterations. The Wilcoxon test yielded a p-value of $p = 0.00195$ for both metrics universally – the minimum possible for $N = 10$ iterations.

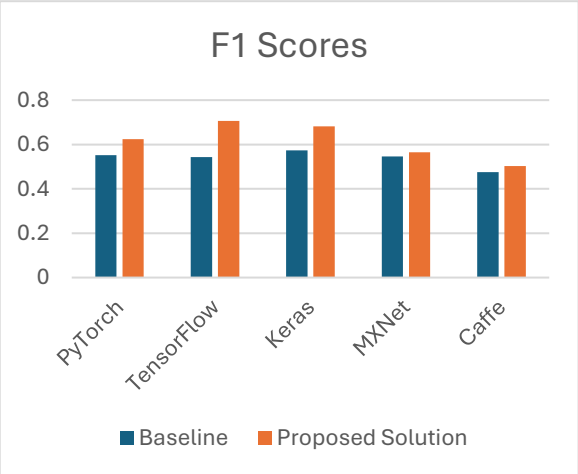

Figure 1: Average F1-Score comparison between the Baseline and Proposed Solution across five datasets

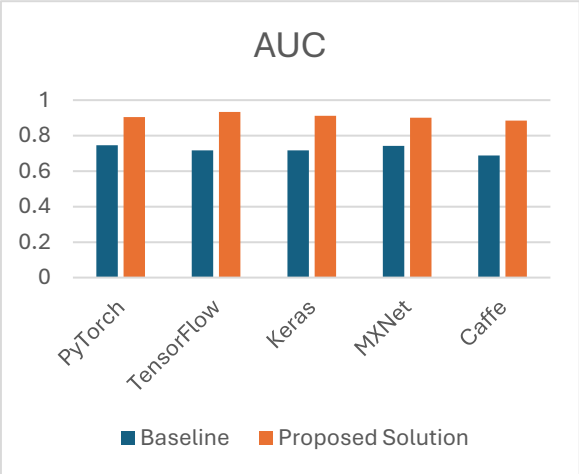

Figure 2: Average AUC comparison between the Baseline and Proposed Solution across five datasets

Achieving a statistical sweep in Accuracy is significant. Because performance bugs constitute only ~16.4% of all the data, models may fall victim to the 'Accuracy Paradox', where they achieve high accuracy by ignoring the minority class. The proposed model's ability to sweep the baseline in Accuracy, while simultaneously utilising class_weight='balanced' to actively identify the minority class, proves that the engineered structural features (code and comment lengths) provided highly relevant

context. Furthermore, the 10/10 AUC sweep exposes the baseline's mathematical flaw: by passing discrete binary predictions instead of continuous probabilities, the baseline distorted the ROC calculation, effectively stripping the metric of its ability to measure the model's confidence ranking.

Analysis of the F1-Scores highlighted nuanced model behaviour across the various projects, despite the proposed solution also routinely beating the baseline F1 Scores across all datasets. For highly structured, log-heavy repositories like TensorFlow, the proposed model achieved a statistically significant F1-Score improvement ($p = 0.00195$), proving the value of separating the lexical and semantic pipelines. However, on the PyTorch dataset – while the proposed model achieved a higher F1-Score – the test returned $p = 0.105$, failing to strictly reject H_0 at the 0.05 threshold. This variance perfectly illustrates the diverse reality of open-source datasets; projects with more informal bug reporting or shorter code snippets reduce the impact of structural metadata. Overall, the proposed model provided universal practical improvements, though its statistical dominance was most present in highly structured environments.

6. Reflection

While the proposed solution successfully outperforms the baseline, it does have a handful of limitations.

The first major limitation is that this model strongly relies on the data being pre-processed. In this case, the provided datasets were already separated into Title, Body, Code, and Comment columns. Bug reports come in a variety of structured and unstructured formats, and so to deploy this in production an additional tool would need to be developed to dynamically extract each section of the bug report.

Another limitation is that despite utilising Latent Semantic Analysis (LSA) to compress vocabulary, the underlying TF-IDF vectorizer remains a ‘Bag-of-Words’ approach. It operates on term frequency and destroys syntactic word order, meaning that while semantic understanding is improved, it can still miss important context.

Finally, the statistical evaluation was inherently limited by the number of iterations during training (N=10). As observed in the PyTorch F1 results, the non-parametric Wilcoxon test lacks the statistical power to confidently reject the null hypothesis with limited samples. Future work on this project would benefit from performing 30 to 50 iterations instead and also explore deep learning methods such as BERT to improve contextual and semantic understanding further.

7. Conclusion

Automated bug triage remains a critical challenge in modern software engineering, particularly for vague and complex performance bugs. This report has identified core flaws in standard baseline approaches, with a focus on multifaceted bug reports being treated as flat text, technical logs being ignored, and classifiers like Gaussian Naive Bayes being poorly chosen. To address this, a structurally aware, multifaceted Logistic Regression architecture was proposed. By explicitly separating the lexical analysis of code snippets from the semantic evaluation of natural language, and integrating structural metadata, the model was able to successfully synthesise the complex nature of bug reports. Rigorous statistical testing across five different bug report datasets confirmed that the proposed solution universally swept the baseline in Accuracy and AUC ($p = 0.00195$), while also delivering robust F1-Score improvements. Ultimately, this demonstrates that moving beyond flat-text analysis into structurally aware feature engineering provides a highly effective, explainable, and intelligent software engineering tool for automated bug triage.

8. Artifact

Codebase with requirements.pdf, manual.pdf and replication.pdf can be found here:
https://github.com/tanisha-bhogal/ise_coursework.git

Appendix A

Table A1: Full 10-Iteration results for PyTorch

	Baseline	Proposed Solution
Accuracy	0.6199	**0.8914**
Precision	**0.6092**	0.5632
Recall	**0.7465**	0.7125
F1 Score	0.5522	**0.6235**
AUC	0.7465	**0.9041**

Table A2: Full 10-Iteration results for TensorFlow

	Baseline	Proposed Solution
Accuracy	0.5617	**0.8758**
Precision	0.6397	**0.6414**
Recall	0.7164	**0.7899**
F1 Score	0.5434	**0.7063**
AUC	0.7164	**0.9330**

Table A3: Full 10-Iteration results for Keras

	Baseline	Proposed Solution
Accuracy	0.5955	**0.8664**
Precision	0.6428	**0.6716**
Recall	**0.7163**	0.7023
F1 Score	0.5737	**0.6823**
AUC	0.7163	**0.9117**

Table A4: Full 10-Iteration results for MXNet

	Baseline	Proposed Solution
Accuracy	0.6125	**0.8962**
Precision	**0.6076**	0.5710
Recall	**0.7423**	0.5643
F1 Score	0.5456	**0.5642**
AUC	0.7423	**0.9012**

Table A5: Full 10-Iteration results for Caffe

	Baseline	Proposed Solution
Accuracy	0.5259	**0.8845**
Precision	**0.5839**	0.4914
Recall	**0.6873**	0.5683
F1 Score	0.4752	**0.5029**
AUC	0.6873	**0.8851**

References

[1] Antoniol et al. (2008), *Is it a Bug or an Enhancement? A Text-based Approach to Classify Change Requests*. Available at: https://dl.acm.org/doi/epdf/10.1145/1463788.1463819 (Accessed: 15th April 2026)

[2] Javed et al. (2012), *An Automated Approach for Software Bug Classification*. Available at: https://ieeexplore.ieee.org/abstract/document/6245635 (Accessed: 17th April 2026)

[3] Nistor et al. (2013), *Discovering, Reporting and Fixing Performance Bugs*. Available at: https://ieeexplore.ieee.org/document/6624035 (Accessed: 15th April 2026)

[4] Chawla et al. (2014), *Automatic Bug Labelling using Semantic Information from LSI*. Available at: https://ieeexplore.ieee.org/abstract/document/6897203 (Accessed: 15th April 2026)

[5] Ramay et al. (2019), *Deep Neural Network-Based Severity Prediction of Bug Reports*. Available at: https://ieeexplore.ieee.org/abstract/document/8685106 (Accessed: 15th April 2026)

[6] Mani et al. (2019), *DeepTriage: Exploring the Effectiveness of Deep Learning for Bug Triaging*. Available at: https://dl.acm.org/doi/abs/10.1145/3297001.3297023?__cf_chl_tk=GXxBTppyFi.jWFYeH1JocJppdzcXI8Fl4wm6_C0oxSY-1776162230-1.0.1.1-oJiC5cZCVUh4ccmvII3skSU0Psgx6lO5K9uVcA7XD7E (Accessed: 15th April 2026)

[7] Long et al. (2022), *Multifaceted Hierarchical Report Identification for Non-Functional Bugs in Deep Learning Frameworks*. Available at: https://ieeexplore.ieee.org/abstract/document/10043235 (Accessed: 15th April 2026)